

Master PRD & Technical Architecture - AI Fitness

Freemium

Versione: 3.0 (Versione "Spiegata" per i Founder) **Target:** Team di sviluppo (2 Ingegneri Informatici Part-Time, supportati da AI/Cursor)

Questo documento contiene l'architettura tecnica definitiva, ma con spiegazioni chiare di ogni termine complesso, così che chiunque nel team possa capire al 100% **perché** stiamo facendo certe scelte.

1. TECH STACK DEFINITIVO (Le tecnologie che useremo)

Componente	Tecnologia	Perché l'abbiamo scelta e Cosa significa
Frontend Mobile (L'app che l'utente vede)	React Native + Expo (Managed)	Cos'è: React Native è un framework per creare app iOS e Android scrivendo un solo codice. Expo è un set di strumenti che facilita enormemente il lavoro. Perché: Permette di sviluppare velocissima. Grazie ai "Config Plugins" di Expo, possiamo usare librerie avanzate (come l'AI visiva) senza dover lottare con il codice nativo di Apple o Google.
Backend / BaaS (Il motore del server invisibile)	Supabase	Cos'è: Un "Backend-as-a-Service". Invece di programmare un server da zero per gestire login e cloud, usiamo Supabase che ci dà tutto pronto e può reggere milioni di utenti. Perché: Costo iniziale nullo e manutenzione zero ("Serverless").
Database (L'archivio dei dati)	PostgreSQL (Incluso in Supabase)	Cos'è: Uno dei database più solidi e affidabili al mondo. Perché: Supporta un formato chiamato JSONB (un modo flessibile per salvare dati), perfetto per salvare le schede generate dall'Intelligenza Artificiale come fossero pacchetti pronti all'uso.
Gestione Pagamenti	RevenueCat	Cos'è: Un sistema che fa da intermediario tra la nostra app e gli App Store per gestire gli abbonamenti (In-App Purchases). Perché: Gestire gli abbonamenti di Apple e Google a mano è un incubo ingegneristico. RevenueCat si assicura che se paghi su iPhone, sei "Pro" anche nel nostro server, gestendo pure le ricevute fiscali.
AI - Generazione Testo (Le schede)	OpenAI API (Modello GPT-4o-mini)	Cos'è: Il cervello testuale dell'app. Perché: Il modello "mini" costa frazioni di centesimo ed è velocissimo. Usiamo il JSON Mode , che costringe l'IA a restituire i dati ordinati perfettamente come vogliamo noi, evitando allucinazioni testuali che farebbero crashare le schermate.
AI - Computer Vision (L'occhio)	MediaPipe (Google) /	Cos'è: La tecnologia che "guarda" i frame video e capisce dove sono braccia e gambe.

che corregge)	Vision Camera	Perché: Lavora in locale sul telefono (On-Device). Non mandiamo un singolo video online. Questo abbate i costi server a ZERO e azzera il rischio di denunce per la privacy (Zero-Video-Upload).
---------------	----------------------	--

2. SCHEMA DEL DATABASE (PostgreSQL)

Ecco come organizziamo i dati nel cloud. Ogni utente o scheda ha un "ID", un codice identificativo unico e inimitabile.

1. Tabella: `utenti` (I nostri iscritti)

- `email` : Email dell'utente.
- `legal_agreed` (Vero/Falso): Un interruttore essenziale. Indica se l'utente ha accettato i rischi. Se è Falso, l'app lo blocca.
- `tier` : "Free" o "Pro".

2. Tabella: `schede` (Gli allenamenti generati)

- `user_id` : Indica a quale utente appartiene la scheda.
- `dati_scheda` : I dettagli (serie, rec, esercizi), salvati come pacchetto dati `JSONB`.

3. Tabella: `esercizi` (Il dizionario che l'app conosce)

- `nome_key` : Il nome tecnico usato dal computer (es. `squat_bilanciere`).

4. Tabella: `progressi_storico` (I risultati inviati alla fine dell'allenamento)

- `reps_totali` : Es. Le 12 ripetizioni valide misurate dall'App visiva.
- `metriche_ai` : Note dall'AI visiva (es. "L'utente spesso non scende a sufficienza").

Row Level Security (RLS) - La nostra Cassaforte

Cos'è: L'RLS (Row Level Security o Sicurezza a livello di Riga) è un "buttafuori" messo alla porta del database. **Cosa fa:** Se un hacker trova un modo di fare richieste al database, la regola RLS dice: *"Controlliamo il tuo badge prima di farti passare. Puoi leggere o inserire dati SOLO nelle righe in cui l'ID corrisponde al tuo account"*. Rende fisicamente impossibile per "Mario" scaricare la scheda o gli storici di "Luigi".

3. ARCHITETTURA DELLA SICUREZZA E FLUSSO LEGALE

Il Login (OAuth)

Useremo solo i pulsanti "**Accedi con Apple**" e "**Accedi con Google**". **Perché:** Evitiamo di far creare password (meno attrito). Inoltre, se non salviamo noi le password nei nostri server, non possono rubarcele in caso di databreach. Ci appoggiamo alla sicurezza dei colossi tech.

Disclaimer di Responsabilità ("Nessun Medico Qui")

Trattando di salute ed IA, dobbiamo svincolarci da cause legali (Liability). La logica:

1. Registrazione: l'utente si logga con Google. Nel database, creiamo l'account e impostiamo `legal_agreed = FALSO`.
2. L'app rileva il "Falso" e mostra una schermata Modale (un muro) non chiudibile.

3. Testo: "Questa AI è sperimentale. Non sostituisce un medico. Analizzeremo i tuoi movimenti sul tuo telefono e non salveremo i video originali per la tua privacy. Se ti infortuni, è sotto la tua totale responsabilità."
 4. Appena l'utente mette la spunta "Accetto e Proseguo", il server viene aggiornato, il campo diventa "VERO", e l'app fa finalmente scomparire il muro.
-

4. DATA FLOW E INTEGRAZIONE LLM (Come chiediamo all'IA)

L'app sul telefono **non** parlerà direttamente con ChatGPT (altrimenti un hacker potrebbe rubarci la password nascosta nell'app e usarla a spese nostre). L'app chiederà al nostro server (Supabase) di farlo per lei con sicurezza.

Il System Prompt (Istruzioni di Rigore per l'IA) Diremo a GPT-4o-mini esattamente questo: *"Sei un Personal Trainer d'élite. Rispondi TASSATIVAMENTE sotto forma di codice JSON puro, non usare testo normale o chiacchiere. La tua risposta deve obbligatoriamente contenere: "nome del programma", e una lista raggruppata per giorni, ogni giorno dovrà avere il nome degli esercizi abbinati a interi che rappresentano Serie, Ripetizioni e Secondi di Recupero. Tieni conto del livello inserito dall'utente."*

Rispondendo in formato JSON netto, le variabili scendono nel telefono e l'app le impagina automaticamente con i cerchi, i colori e i testi corretti, senza bug visivi.

5. LOGICA AI VIDEO (Macchina a Stati - State Machine)

Cos'è una "State Machine" (Macchina a Stati)? Immagina un semaforo: non può mai essere giallo e rosso assieme, o è uno o l'altro. Il nostro riconoscimento visivo per uno `Squat`, che misura il corpo a 30 fotogrammi al secondo tramite l'uso della geometria e della trigonometria, vive costantemente in uno di questi "stati":

1. **Stato: RICERCA_CORPO**
 - Cosa fa: Il radar dell'AI (MediaPipe) dice "non sono sicuro di dove sia il ginocchio".
 - Sullo schermo: Spesso rosso, scritta *"Allontanati per inquadrare i piedi"*.
2. **Stato: IN_POSIZIONE**
 - Cosa fa: Le telecamere vedono anche le caviglie. La geometria dice che le gambe sono dritte (angolo di 180 gradi).
 - Sullo schermo: Schermo Verde, "Pronti, via!".
3. **Stato: ESECUZIONE_DISCESA**
 - Cosa fa: L'angolo tra anca e ginocchio sta diminuendo.
 - Sullo schermo: Cerchio progressivo che si riempie.
4. **Stato: PUNTO_CRITICO (La profondità corretta)**
 - Cosa fa: La geometria rileva che l'anca è andata fisicamente sotto la riga virtuale del ginocchio (squat "sotto il parallelo").
 - Azione: Il telefono emette una piccola vibrazione fisica (Feedback Aptico) per dire all'utente "Ok, hai toccato il traguardo, ora risali!".
5. **Stato: ESECUZIONE_SALITA**
 - Cosa fa: Le ginocchia tornano dritte. Se prima c'era stata la vibrazione del punto critico, calcoliamo una Ripetizione Valida `+1`. Altrimenti (mezzo squat) diciamo vocalmente *"Vai più giù"*.
6. **Stato: SERIE_FINITA**

- Cosa fa: Abbiamo raggiunto target stabilito nella scheda generata e chiudiamo la telecamera. Inviemo 5kb di testo al cloud ("Esercizio concluso: 10/10 buone").

6. ROADMAP DI IMPLEMENTAZIONE (Task Pronti per Cursor)

Questa è un'agenda granulare: voi founder potete copiare e incollare questi incarichi letteralmente nel box chat di Cursor per farvi accompagnare nella generazione del codice.

Fase 1: Tutta la Base e il Database

- **A.** "Cursor, Inizializza un progetto React Native / Expo in Typescript. Predisponi l'Expo Router per la navigazione delle schermate."
- **B.** "Cursor, collegaci all'SDK di Supabase usando le chiavi ambiente, crea un file `database.types.ts`."
- **C.** A mano: Aprire il sito web di Supabase, schiacciare il tasto nuovo database, e incollare gli script per generare `utenti`, `schede` ecc. Forzando subito le chiavi di RLS.

Fase 2: Autenticazione Muro di Cemento Legale

- **A.** "Cursor, voglio integrare `@react-native-google-signin`. Scrivimi la schermata Login con il bottone Google, intercetta il token e passalo a Supabase Auth."
- **B.** "Cursor, fai lo stesso per `expo-apple-authentication`."
- **C.** "Cursor, se login avvenuto, crea un check sul campo `legal_agreed`. Se falso, mostrami la full page modale di disclaimer come definita nel PRD. Non deve farti navigare via finchè la checkbox non è spuntata e Supabase aggiornato."

Fase 3: Soldi e AI Generativa

- **A.** Comprare un server (Edge function) su Supabase gratis. "Cursor, scrivimi un Deno Edge Function che prende un token Firebase, chiama l'API di OpenAI GTP-4o-mini con il System Prompt XYZ, e restituisce il JSON formattato all'app mobile."
- **B.** "Cursor, installa e usa libreria `react-native-purchases` per RevenueCat. Se voglio creare una nuova scheda allenamento, controlla i miei Entitlements (sblocchi). Se sono 'Free', reinderizzami sulla magnifica pagina d'acquisto."

Fase 4: La Computer Vision (Lavori manuali on-device hard)

- **A.** "Cursor, installa `react-native-vision-camera` e `react-native-fast-tflite`. Assicuratevi di aggiungere i permessi privacy per la fotocamera nei config file iOS e Android."
- **B.** "Cursor, fammi un componente `CameraView`."
- **C.** "Cursor, inietta nel feed MediaPipe. Costruiamo il modulo Typescript della 'State Machine' dello squat e usiamo `react-native-reanimated` per disegnare i loghi, linee o rettangoli in fluidità assoluta sopra al video utente, in tempo reale, ascoltando l'angolo calcolato."